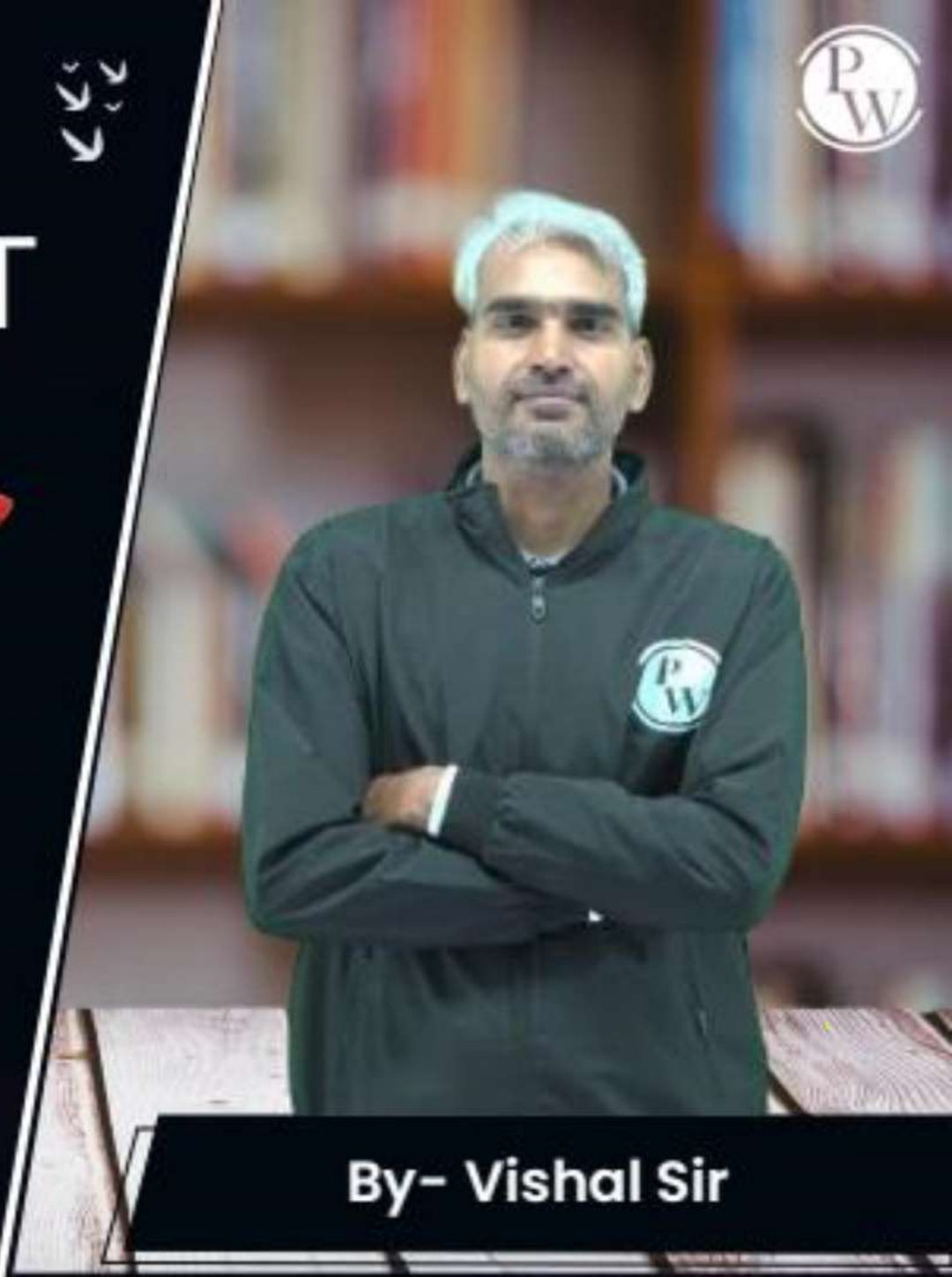
Computer Science & IT



Database Management System

File organization And Indexing

Lecture No. 05



By- Vishal Sir

Recap of Previous Lecture



- **Topic** Structure of lean node and internal node of B tree
- **Topic** Structure of lean node and internal node of B+ tree
- **Topic** Insertion into B tree
- **Topic** Deletion from B tree

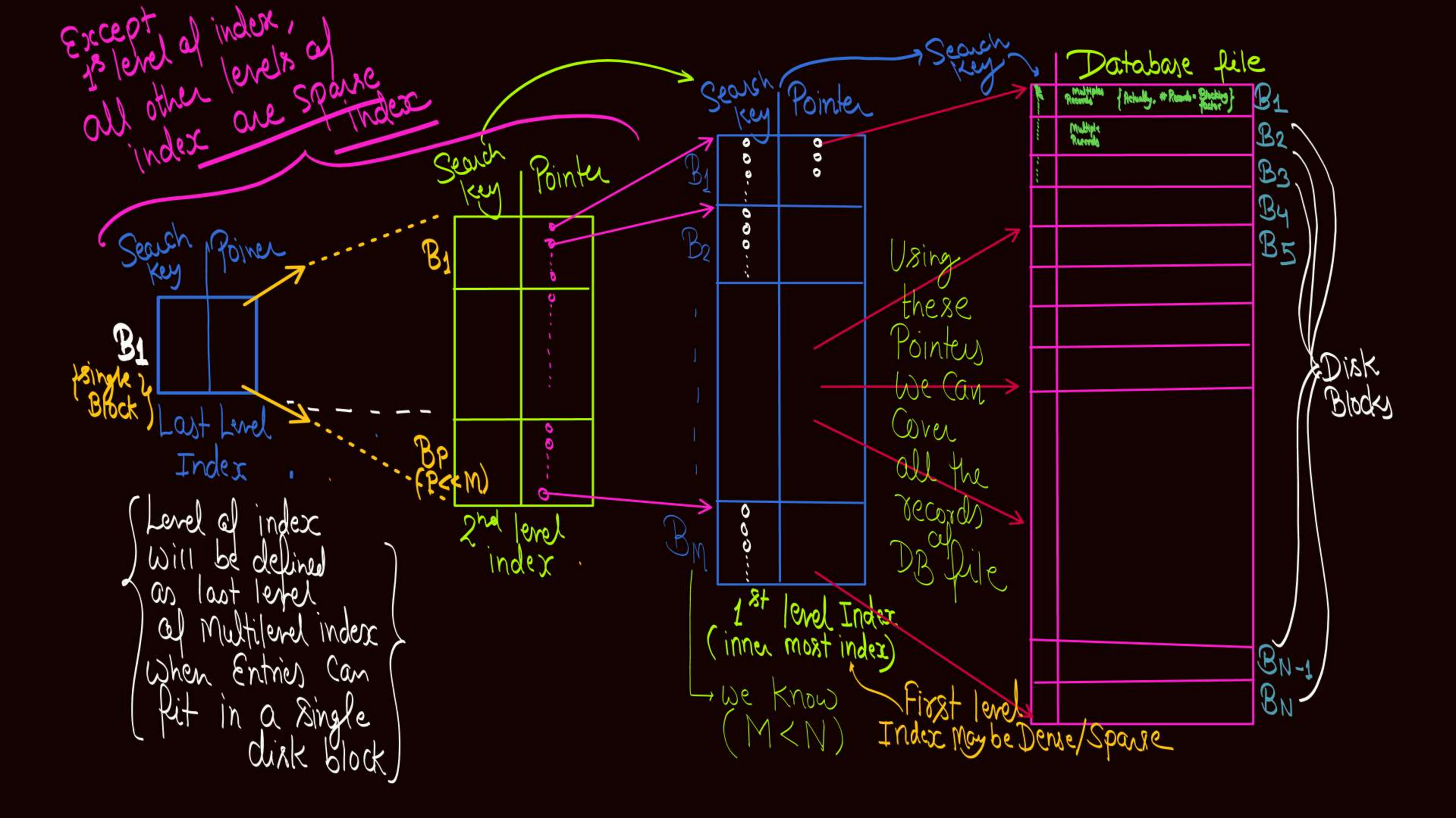
Slide

Topics to be Covered



- Topic: Deletion from B tree
- Topic: Insertion into B+ tree
- Topic: Deletion from B+ tree

Slide


Except 1st level of index, all other levels of index are sparse Index
Search Key
Pointer
B1
(single Block)
Last Level Index
Bp (P<M)
2nd level index
BM
1st level Index (inner most index)
we know (M<N)
Search Key
Database file
Multiple Records
{Actually, # Records = Blocking factor}
B1
B2
B3
B4
B5
BN-1
BN
Disk Blocks
Using these Pointers we can Cover all the records of DB file
First level Index may be Dense/Sparse
{Level of index will be defined as last level of Multilevel index when Entries can fit in a single disk block}

* Multi-level index can improve significantly in terms of IO Cost.

* IO Cost with Multi-level Index :-

In a multi-level index if there are "S" levels of index then,

IO Cost (to access a record) = No. of levels in Multi-level index + 1

- No. of levels in Multi-level index: from each level of index '1' block will be transferred
- 1: This is w.r.t. block of the database file to access the record based on addr provided by inner most level of index

IO Cost = S + 1

Using multi-level index

IO Cost = No. of levels + 1

- for all accesses.

{i.e. Best case, worst case, Avg case always same}

In multi-level Index :-

1. 1st level of index can be dense / sparse and other levels of index are always sparse.
2. IO Cost = No. of levels + 1

Note: * The multi-level index that we have just discussed is suitable only for static database
{i.e. insert/delete/update op^n are very rare}

. If database is dynamic {i.e. insert/delete/update are frequent} then after every such operation we will have to restructure all the levels of multi-level index; and it will be time consuming

i.e. if database is dynamic then significant portion of time will be wasted in maintaining the index itself.

* Above multi-level index is not suitable for Dynamic database,

∴ We use dynamic multilevel index tree for dynamic database

- B tree
- B+ tree

} These are self-balancing multi-way search tree.



Topic : Dynamic Multi-level Index

- B tree
- B+ tree

All leaf nodes will always be at same level
{ ∴ Balanced Search tree }

{ B tree & B+ tree are Multi-way Search tree }



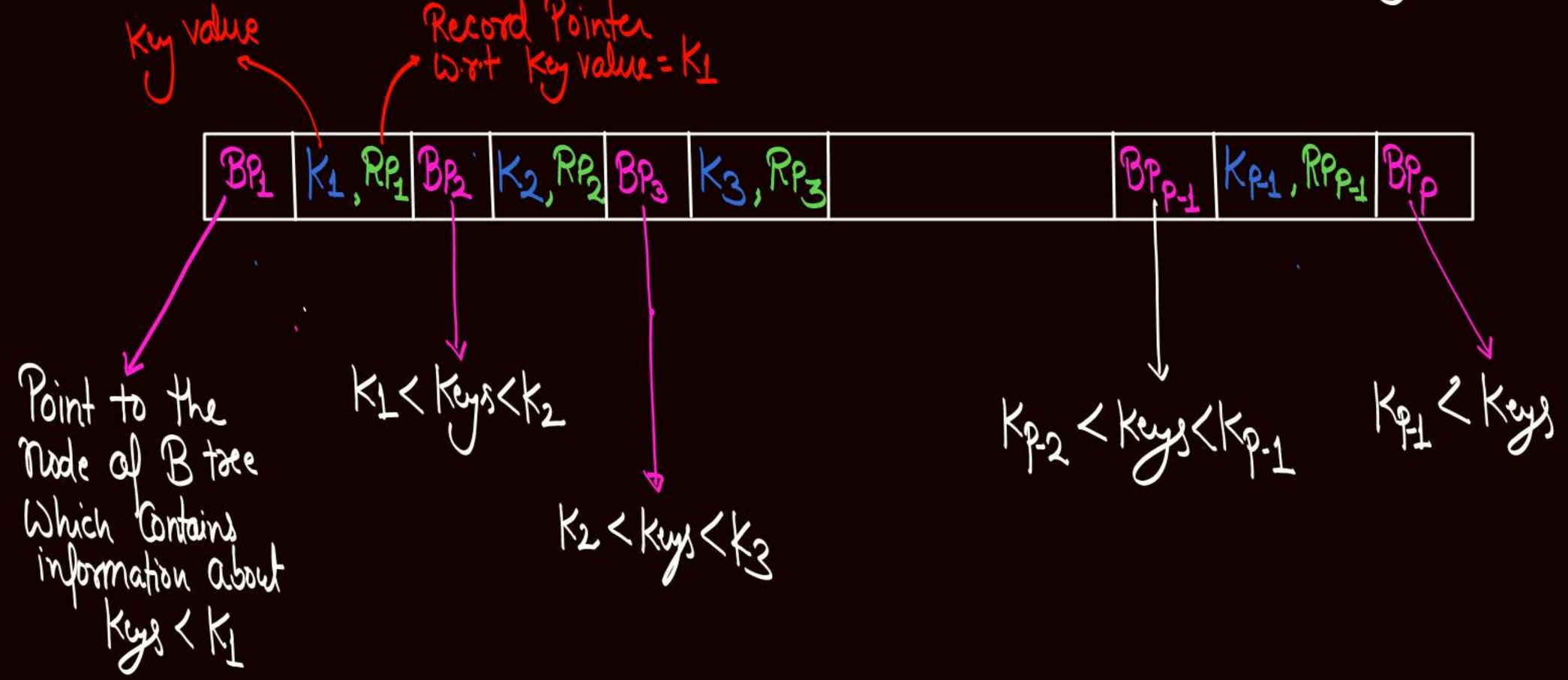
Topic : B Tree

- **Order of a node of B tree :-** Order of a node of B tree is defined as maximum number of Child pointer a node can have

* Consider a B tree of order = p { i.e, Max No. of Child pointer = p ∴ Max. no. of keys = (p−1)

Structure of an internal node of B tree

All keys within the node will always be in ascending order.



Record Pointer (Rp)

Record pointers are used to point the record in the database file corresponding to the "key value"

Block Pointers (Bp)

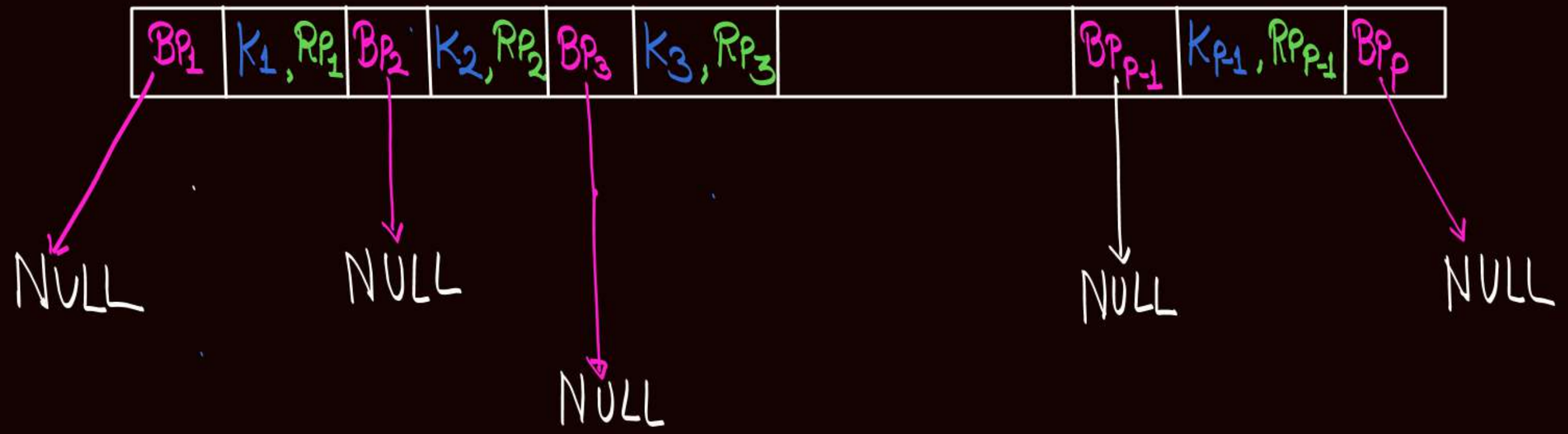
Block pointers are used to point the index Blocks of the B tree at lower level.

{These are also known as Child pointer/ tree pointer/ node pointer}

* Consider a B tree of order = p { i.e, Max No. of Child pointers = p ; ∴ Max. no. of keys = (p−1)

Structure of an leaf node of B tree

All keys within the node will always be in ascending order.



Note: In B tree, address of the record (record pointer) w.r.t. 'key' is available in leaf node as well as in internal node

∴ Structure of B tree is different from normal multi-level index in which record pointers are available only at the leaf nodes (i.e, inner most level of index)

Note:- Each node of B tree is a disk block,

∴ Whatever we want to store in one node of B tree that must fit into a single disk block.

* Consider a B tree of order = p { i.e., Max No. of Child pointers = p
∴ Max no. of keys = (p-1)

It is one B tree node ⇒



And it must fit into a single block ⇒

$$\therefore \left(P * \text{Block Pointer size} + (P-1)\,\text{Search key size} + (P-1)\,\text{Record Pointer size}\right) \leq \text{Disk Block}$$

$$P * \text{Block Pointer Size} + (P-1) * \{\text{Search key size} + \text{Record pointer size}\} \leq \text{Disk Block size}$$

Using this eqn, we can find the maximum possible value of p



Topic : B Tree

Consider a relations R with a key filed K. A B-tree of order P is used to access structures on K. Where 'P' denotes the maximum Number of tree pointers in a B-tree index node. Assume that key (k) is 30 bytes long disk block size 1024 byte. Each data pointer size P_D is 12 bytes long & each block pointer P_B is 10 byte long. In order for each B-Tree Node to fit in a single disk block, the maximum value of P is-

$$(P*10 + (P-1)*(30+12)) \leq 1024$$

$$52P \leq 1024+12+30$$

$$P \leq \frac{1066}{52}$$

$$P \leq 20.56$$

$$\boxed{P \leq 20}$$

A 19

B 20

C 21

D 22

Note :- In a B tree of order = P

1. Maximum number of Child pointer (Block Pointer) a node of B tree Can have = P
2. Maximum number of Keys a node of B tree Can have is = (P-1)
3. Minimum number of Child pointer (Block Pointer) the root node of B tree Can have = 2
4. Minimum number of Keys the root node of B tree Can have = 1
5. Minimum number of Child pointer (Block Pointer) a non-root node of B tree Can have = $\lceil \frac{P}{2} \rceil$
6. Minimum number of Keys a non-root node of B tree Can have = $\lceil \frac{P}{2} \rceil - 1$

A node of B tree must be atleast half filled.

#Q. a) Find the minimum number of nodes required in B tree of order = p and height = h

b) Find the minimum number of child pointers required in B tree of order =p and height = h

c) Find the minimum number of keys required in B tree of order=p and height = h

d) Find the maximum number of nodes possible in B tree of order = p and height = h

e) Find the maximum number of child pointers possible in B tree of order = p and height = h

f) Find the maximum number of keys possible in B tree of order = p and height = h

Height / Level	Min. No. of nodes	Min. No. of Child Pointer	Min. no. of Keys required	Max. no. of nodes	Max. no. of Child Pointer	Max. no. of Keys possible
0 / 1 (Level = 1 + height)	1	2	1	1	p	$(p-1)$
1 / 2	$2 = 2*\lceil \frac{p}{2} \rceil^0$	$2*\lceil \frac{p}{2} \rceil$	$2*(\lceil \frac{p}{2} \rceil - 1) = 2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^0$	p	$p*p = p^2$	$p*(p-1)$
2 / 3	$2*\lceil \frac{p}{2} \rceil = 2*\lceil \frac{p}{2} \rceil^1$	$2*\lceil \frac{p}{2} \rceil*\lceil \frac{p}{2} \rceil = 2*\lceil \frac{p}{2} \rceil^2$	$2*\lceil \frac{p}{2} \rceil*(\lceil \frac{p}{2} \rceil - 1) = 2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^1$	p^2	$p^2*p = p^3$	$p^2*(p-1)$
3 / 4	$2*\lceil \frac{p}{2} \rceil^2$	$2*\lceil \frac{p}{2} \rceil^2*\lceil \frac{p}{2} \rceil = 2*\lceil \frac{p}{2} \rceil^3$	$2*\lceil \frac{p}{2} \rceil^2*(\lceil \frac{p}{2} \rceil - 1) = 2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^2$	p^3	$p^3*p = p^4$	$p^3*(p-1)$
⋮	⋮	⋮	⋮	⋮	⋮	⋮
h / h+1	$2*\lceil \frac{p}{2} \rceil^{h-1}$	$2*\lceil \frac{p}{2} \rceil^h$	$2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^{h-1}$	p^h	p^{h+1}	$p^h*(p-1)$

Height / Level	Min. No. of nodes	Min. No. of Child Pointers	Min. no. of Keys required	Max. no. of nodes	Max. no. of Child Pointers	Max. no. of Keys possible
0 / 1 (Level = 1 + height)	1	2	1	1	p	$(p-1)$
1 / 2	$2 = 2*\lceil \frac{p}{2} \rceil^0$	$2*\lceil \frac{p}{2} \rceil$	$2*(\lceil \frac{p}{2} \rceil - 1) = 2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^0$	p	$p*p = p^2$	$p*(p-1)$
2 / 3	$2*\lceil \frac{p}{2} \rceil = 2*\lceil \frac{p}{2} \rceil^1$	$2*\lceil \frac{p}{2} \rceil*\lceil \frac{p}{2} \rceil = 2*\lceil \frac{p}{2} \rceil^2$	$2*\lceil \frac{p}{2} \rceil*(\lceil \frac{p}{2} \rceil - 1) = 2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^1$	p^2	$p^2*p = p^3$	$p^2*(p-1)$
3 / 4	$2*\lceil \frac{p}{2} \rceil^2$	$2*\lceil \frac{p}{2} \rceil^2*\lceil \frac{p}{2} \rceil = 2*\lceil \frac{p}{2} \rceil^3$	$2*\lceil \frac{p}{2} \rceil^2*(\lceil \frac{p}{2} \rceil - 1) = 2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^2$	p^3	$p^3*p = p^4$	$p^3*(p-1)$
⋮	⋮	⋮	⋮	⋮	⋮	⋮
h / h+1	$2*\lceil \frac{p}{2} \rceil^{h-1}$	$2*\lceil \frac{p}{2} \rceil^h$	$2*(\lceil \frac{p}{2} \rceil - 1)*\lceil \frac{p}{2} \rceil^{h-1}$	p^h	p^{h+1}	$p^h*(p-1)$

Summation of all these terms will provide the Minimum no. of nodes required in a B tree of Order = p & Height = h

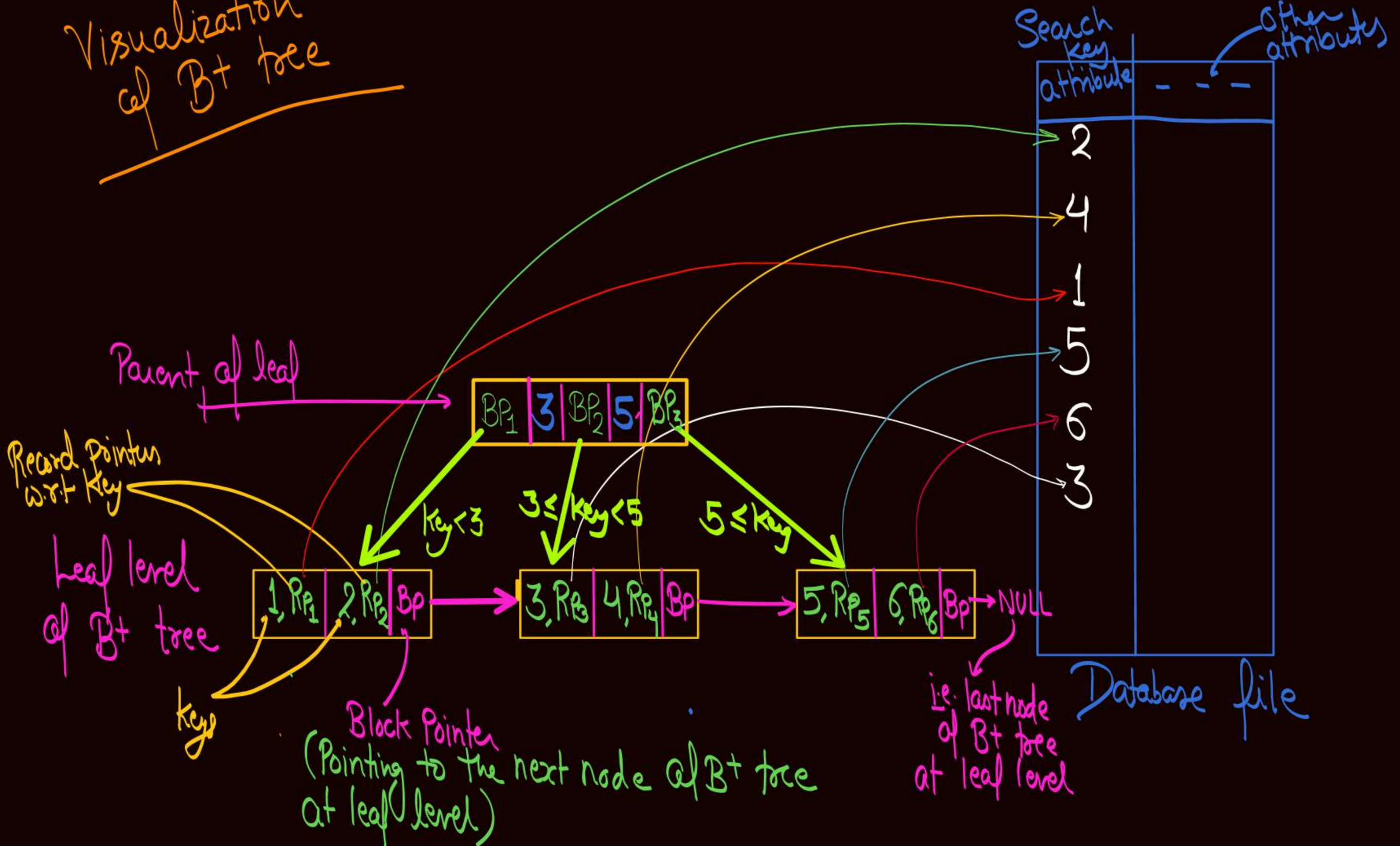
Summation will give Minimum no. of child pointers present in a B tree of Order = p & Height = h



Topic : B+ tree visualization

* Structure of B+ tree is some what similar to the structure of multi-level index.

Visualization of B+ tree





Topic : B+ Tree

- B+ tree is some what similar to normal multi-level index.

- In B+ Tree all keys along with their record pointers are present at leaf nodes of B+ Tree.
- Keys within the node (leaf + internal) of B+ tree must be in ascending order
- In B+ Tree keys in the internal nodes are used only for search operation (they are dummy keys), they are not used to point any record of the database as no record pointer is available in internal nodes of B+ Tree.



Topic : Structure of internal node of B+ Tree







Topic : Order of Internal node of B+ Tree

Order of an internal node of B+ tree is defined as the maximum number of child pointer (block pointer/ node pointer) a B+ tree node can have. (same as B Tree)

- Let order of a node of B+ tree is denoted by 'q'
  - Every information stored in an internal node of B+ tree must fit into a single disk block
  - if order of internal node of B+ tree = q.

then,

$$(q * \text{Block Pointer size} + (q-1) * \text{Search key size}) \leq \text{Disk Block size}$$



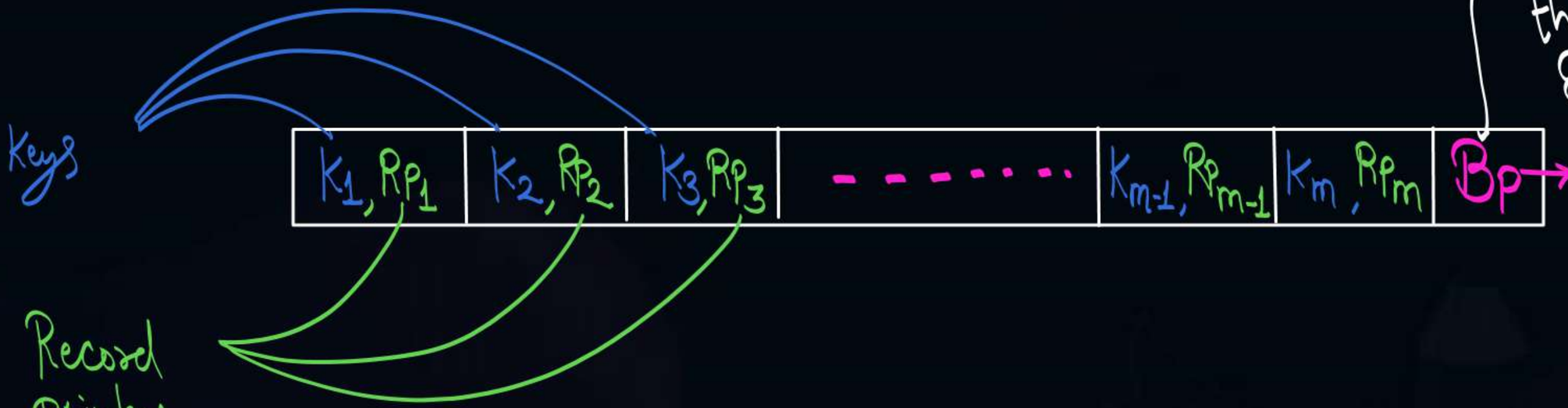
Topic : Order of Internal node of B+ Tree

Let ‘q’ is the order of an internal node of B+ Tree

- Maximum number of child pointer a node can have is = q
- Maximum number of keys a node can have is = q-1
- Minimum number of child pointer a non-root node can have is = $\lceil q/2 \rceil$
- Minimum number of keys a non-root node can have is = $\lceil q/2 \rceil$-1
- Minimum number of child pointer root node can have is = 2
- Minimum number of keys root node can have is = 1



Topic : Structure of leaf node of B+ Tree



In each leaf node of B+ tree there will be Exactly one block pointer, pointing to the next node at the leaf level



Topic : Order of leaf node of B+ Tree

Order of leaf node of B+ tree is defined as the maximum number of keys(record pointers) a B+ tree leaf node can have. (different from B Tree)

Let order of a leaf node of B+ tree is denoted by 'm'

- Every information stored in a leaf node of B+ tree must fit into a single disk block
- If order of leaf node of B+ tree is 'm':

then,

$$\left(m * (\text{Search key Size} + \text{Record Pointer Size}) + 1 * \text{Block Pointer Size}\right) \leq \text{Disk Block Size}$$



Topic : Order of leaf node of B+ Tree

- Let 'm' is the order of a leaf node of B+ Tree
  - Maximum number of keys a leaf node can have is = m
  - Minimum number of keys a non-root leaf node must have is = $\lceil m/2 \rceil$
    - * If leaf node is also a root node, then min. no. of keys allowed = 1
  - There will be exactly one block pointer in each leaf node, it will point to the next leaf node of B+ tree. (because of this pointer range query is more efficient in B+ tree)



Topic : NOTE

- If only one order is given in the question, then we will use the same order for both leaf node as well as for internal node.

Insertion & Deletion in/from B tree

★ Insertion in B tree

{ Insertion will be performed only at leaf level }

→ Let the order of a node of B tree = P

① Search for the leaf node that can hold the key which needs to be inserted.

② If leaf node is not already full {i.e. Contains less than (P-1) keys}, then simply insert the key in the node (in ascending order)

③ If leaf node is already full {i.e. Contain Exactly (P-1) keys} then because of insertion of the new key there will be overflow. In this case we will insert the key in ascending order, then we will split the node into parts, and promote the median position key to the parent node.

3 @ :- If parent node is not already full, then it will accomodate the promoted key, and no further action is required.

3 ⓑ : If parent node is already full, then split the Parent node into two parts and promote the median position key to the parent node of current node,

We will keep repeating this process until the parent node can accomodate the promoted key, this may go upto root node, if root node is also full, then split the root node as well and median position key from root node will become the new root node {i.e. Height of tree may increase by 1}

eg:- Let order of B tree = 4, and keys to be inserted are

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25

in the same order,

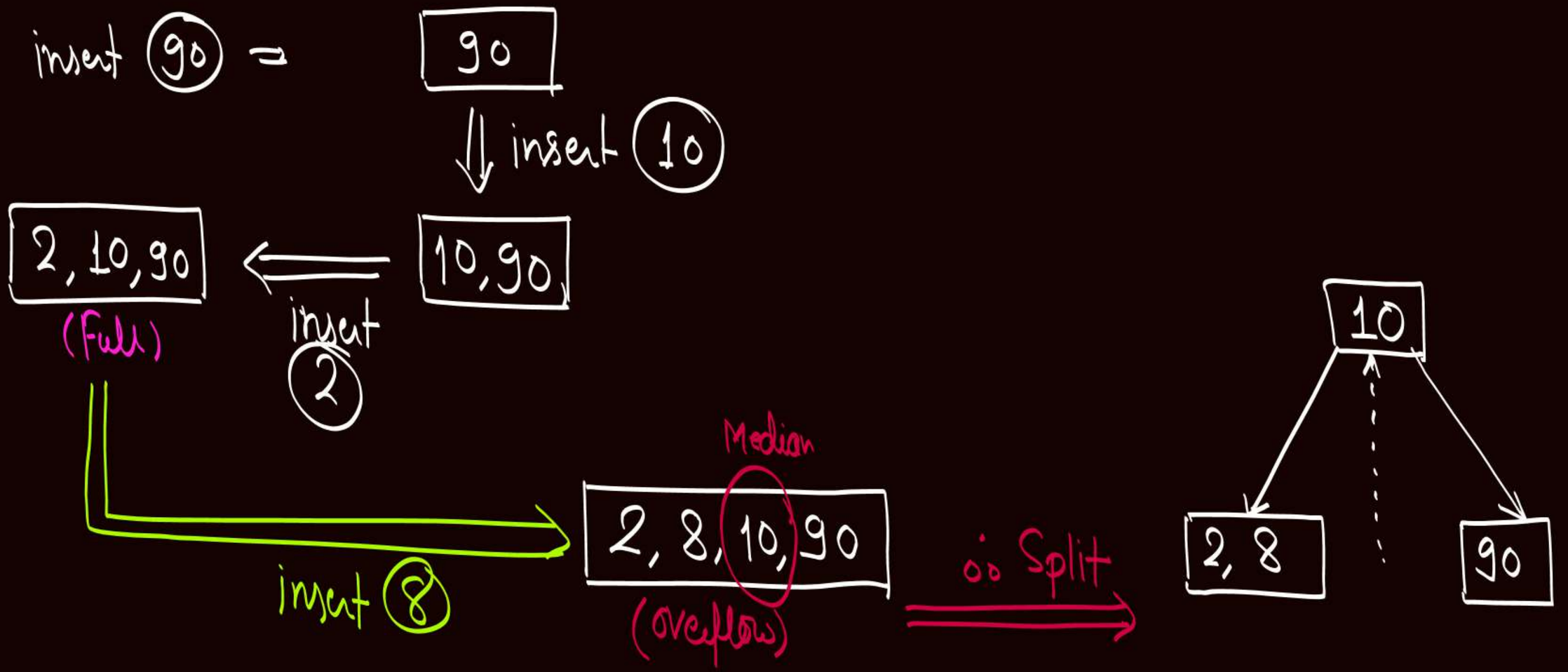
Construct the B tree

- Maximum no. keys a node can hold = 4-1 = 3

eg:-

• Maximum no. Keys a node can hold = 4-1 = 3

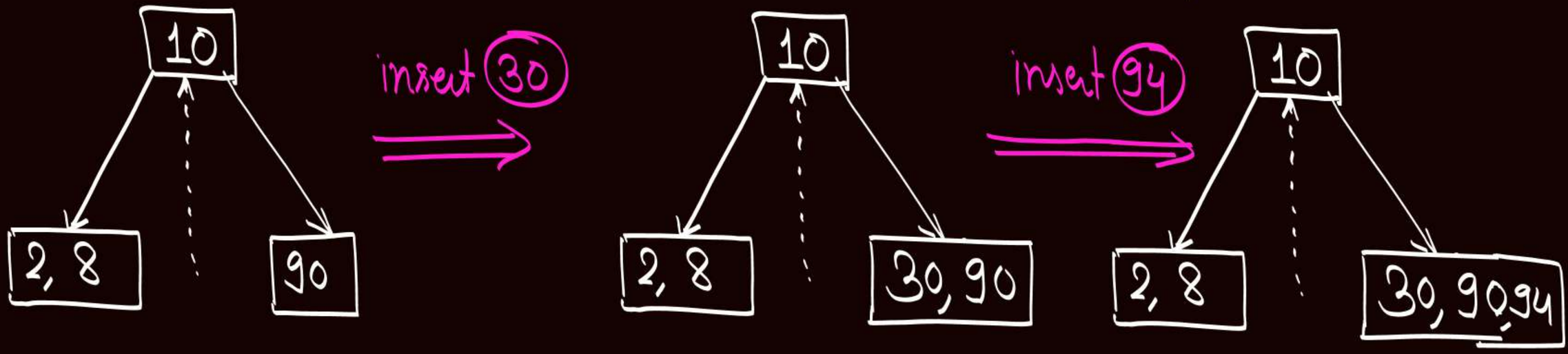
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



eg:-

* Maximum no. keys a node can hold = 4-1 = 3

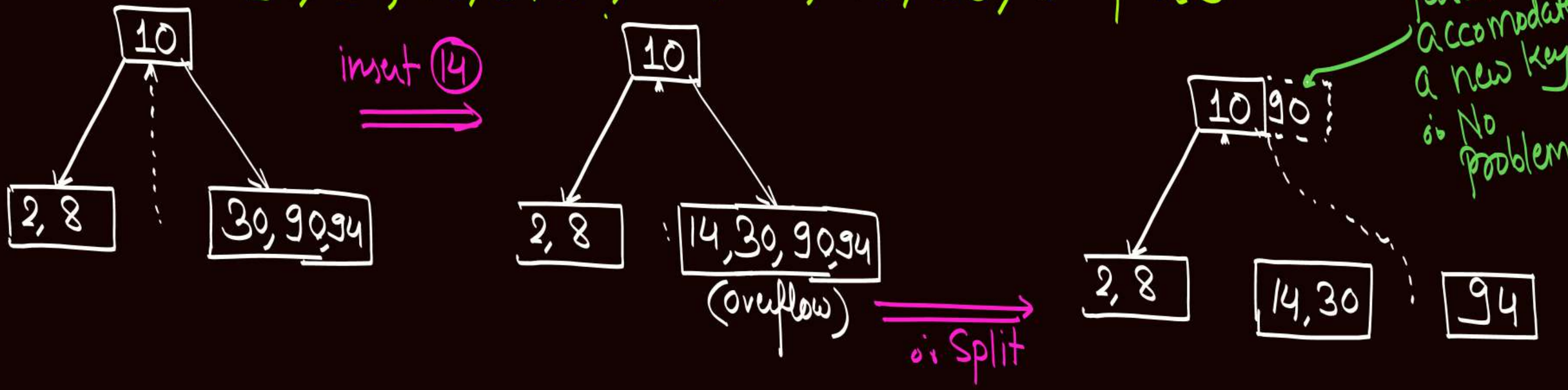
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



eg:-

- Maximum no. keys a node can hold = 4-1 = 3

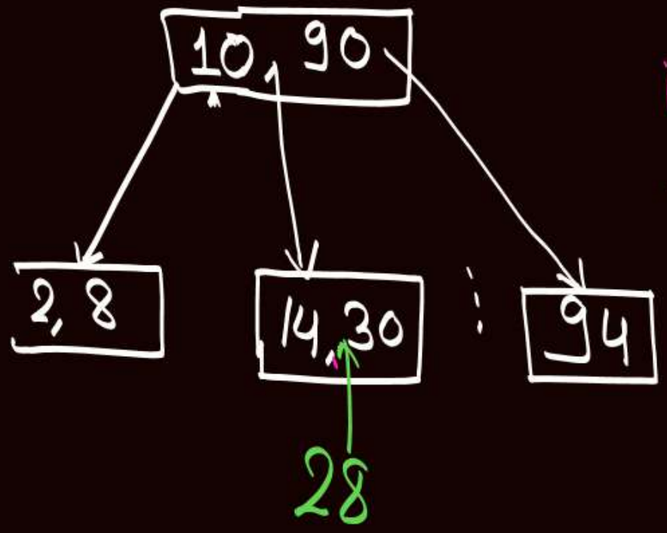
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



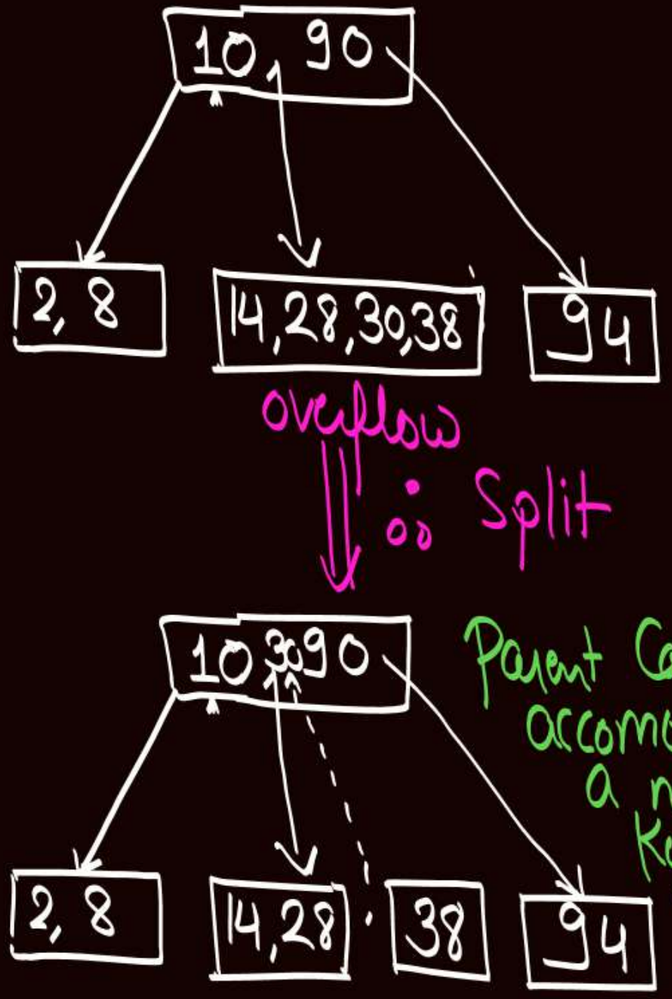
eg:-

- Maximum no. keys a node can hold = 4-1 = 3

90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25

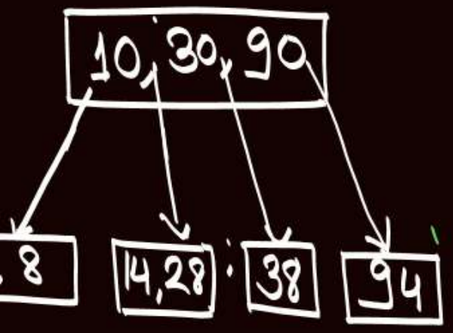


insert (28) & (38)

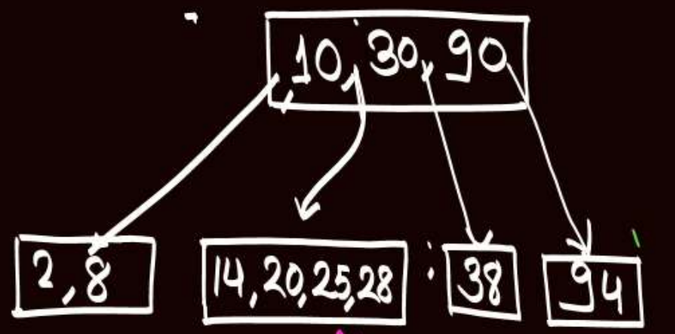


- Maximum no. keys a node can hold = 4-1 = 3

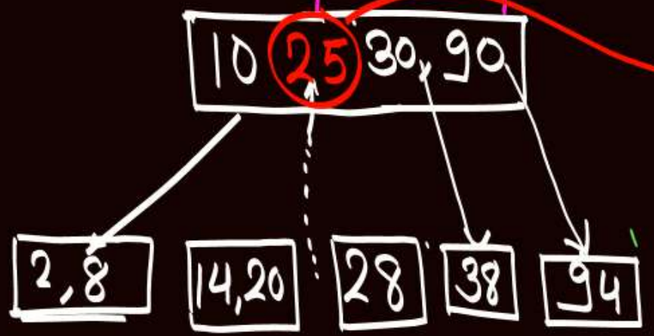
90, 10, 2, 8, 30, 94, 14, 28, 38, 20 & 25



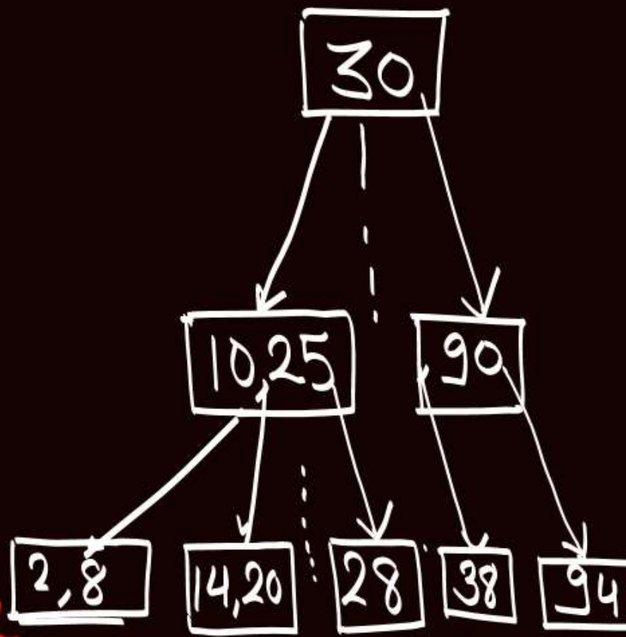
insert (20) & (25)



overflow ∴ Split



Parent is already full
∴ Split Parent node



Deletion from B tree :→ {Key to be deleted may be Present in a leaf node or in an internal node}

If key to be deleted is present in an internal node then replace it with in-order predecessor {i.e, with largest key in left sub-tree} or inorder successor {i.e, with smallest key in right sub-tree}, and then delete the key from leaf node using the following approach.

① If leaf node from which the key needs to be deleted contains more than the minimum number of keys required $\left\{\text{i.e. No. of keys} > \left(\left\lceil \frac{p}{2} \right\rceil - 1\right)\right\}$, then simply delete the key from lead node, no further action is required.

(2) If leaf node from which the key needs to be deleted contains Exactly the minimum number of keys required {i.e. No. of keys $= \left(\left\lceil \frac{p}{2} \right\rceil - 1\right)$}, then after deletion this node will become deficient node {i.e. underflow}

2-(a) → Check with the immediate sibling if they can help. {first check with left sibling, then check with right sibling}

If any one of the immediate sibling can help then borrow (redistribute) key via parent.

(i) Borrow through left sibling : Largest key from the left sibling will be promoted to the parent node, and corresponding key from the parent node will be transferred to the deficient node

(i) Borrow through right sibling : Smallest key from the right sibling will be promoted to the parent node, and corresponding key from the parent node will be transferred to the deficient node

2-(b) → Check with the immediate sibling if they can help. { first check with left sibling, then check with right sibling }

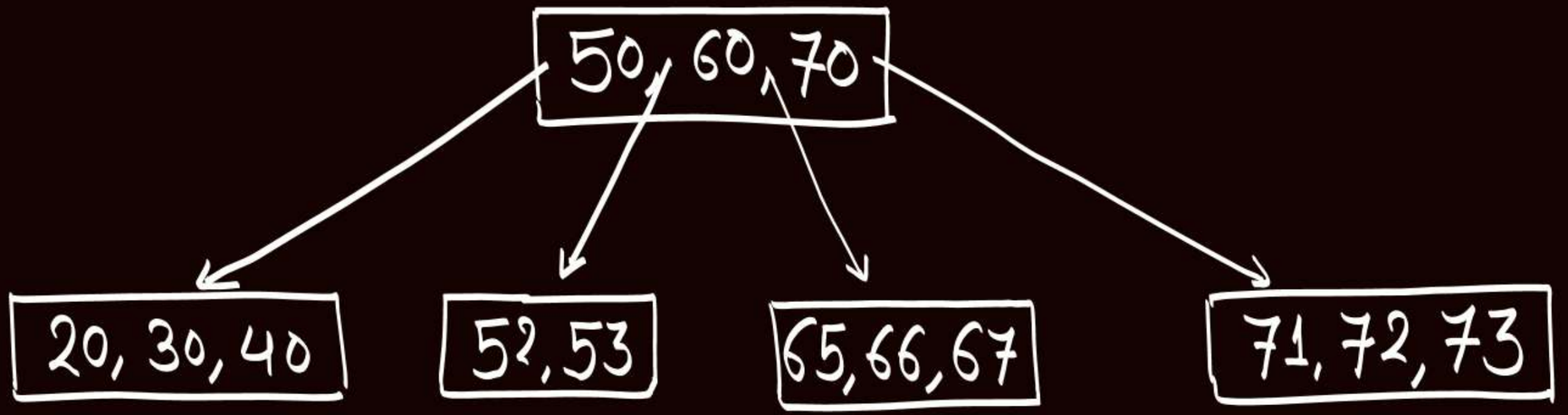
If none of the immediate sibling can help then merge the deficient node with one of its sibling along with one key from parent node.

As a result of this process the parent node may become deficient node

- if parent node does not become deficient then stop.
- if parent node becomes deficient then it will ask its immediate sibling for help and same will follow
  - it may go upto root node, if root node also becomes deficient, then height of tree will reduce by '1'!

H.W.

Q: Consider following B tree of order = 5.

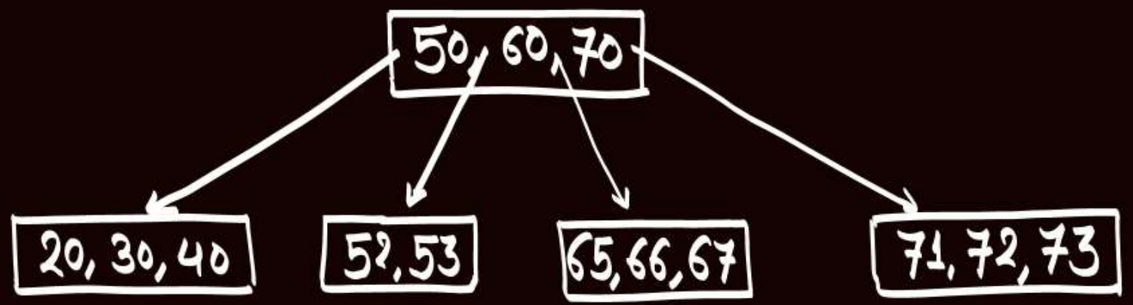


Delete the following keys from above B tree in the same order,

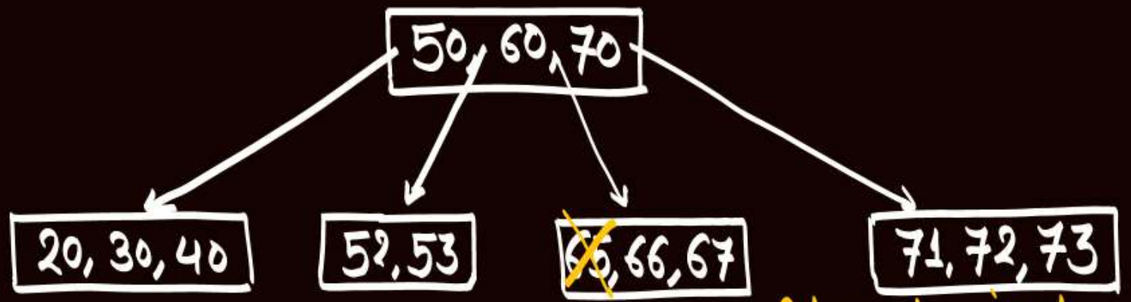
65, 53, 66, 52, 60

H.W.

Q: Consider following B tree of order = 5.



⇓ delete (65)



{ Already in the leaf node
and leaf node contain
More than Min. no. of keys required
so simply delete

order = 5

Min. of key
a non-root node
must have = $\lceil \frac{5}{2} \rceil - 1 = 2$

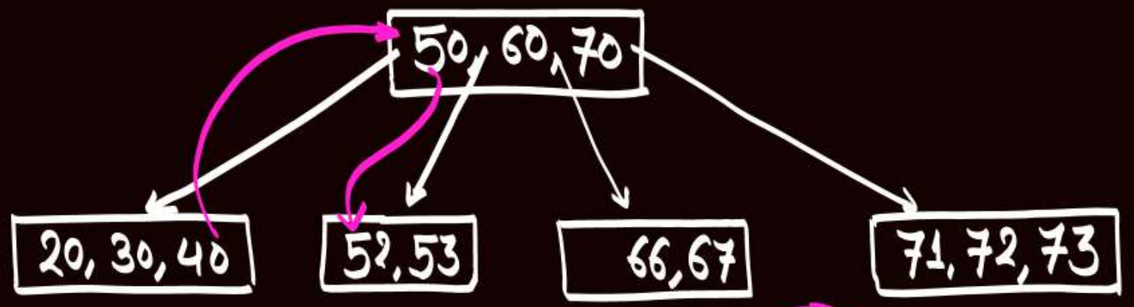
Min. of key
a root node
must have = 1

Delete the following keys from above B tree in the same order,

65, 53, 66, 52, 60

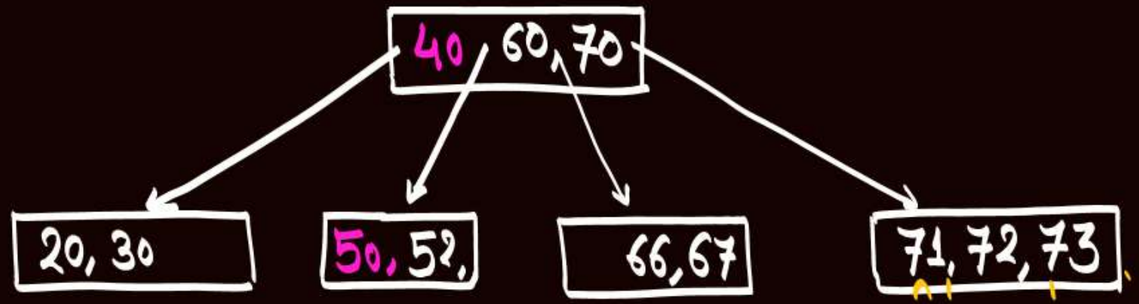
H.W.

Q: Consider following B tree of order = 5.



⇓ delete (53)

{ Already at leaf node, but leaf node contain Min no of keys {left sibling can help}



order = 5

Min. of key a non-root node must have = $\lceil \frac{5}{2} \rceil - 1 = 2$

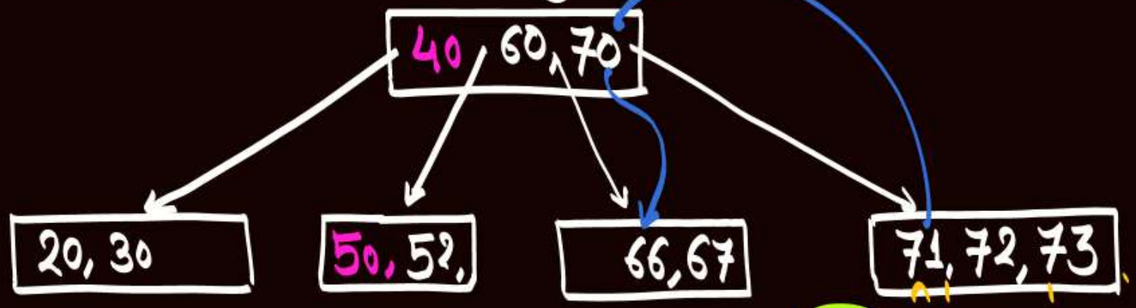
Min. of key a root node must have = 1

Delete the following keys from above B tree in the same order,

65, 53, 66, 52, 60

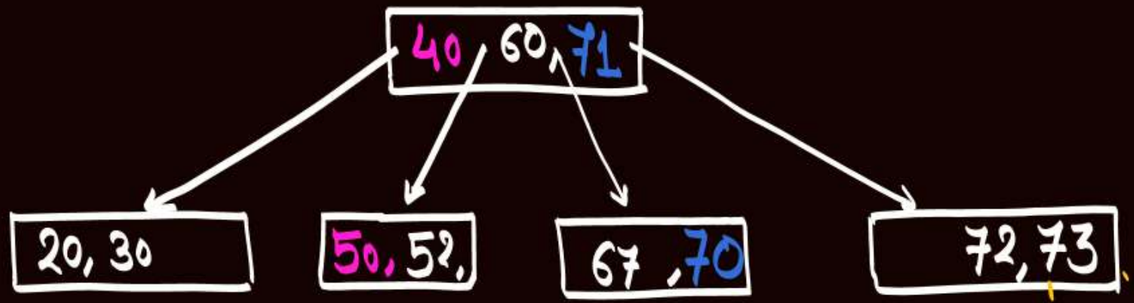
H.W.

Q: Consider following B tree of order = 5.



⇓ delete (66)

{ Already at leaf node, but leaf node contain Min no of keys { Right sibling can help }



order = 5

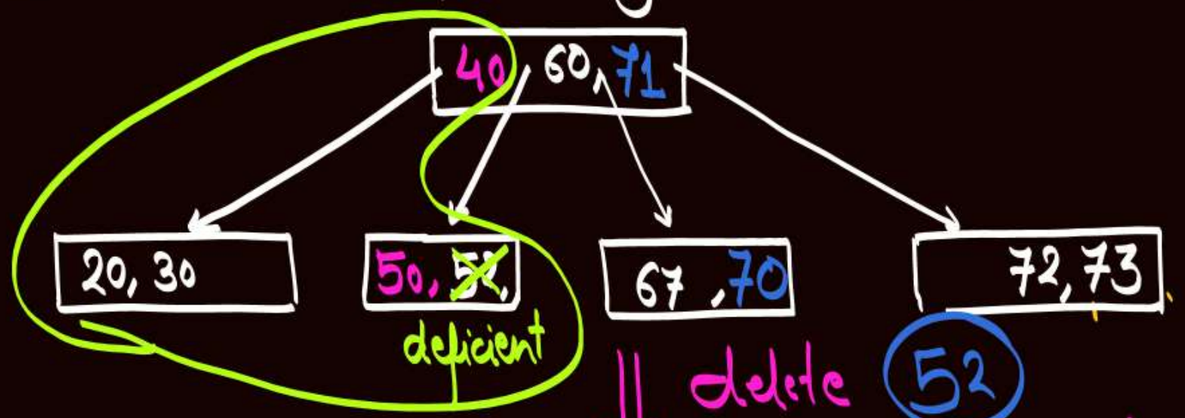
Min. of key a non-root node must have = $\lceil \frac{5}{2} \rceil - 1 = 2$

Min. of key a root node must have = 1

Delete the following keys from above B tree in the same order,

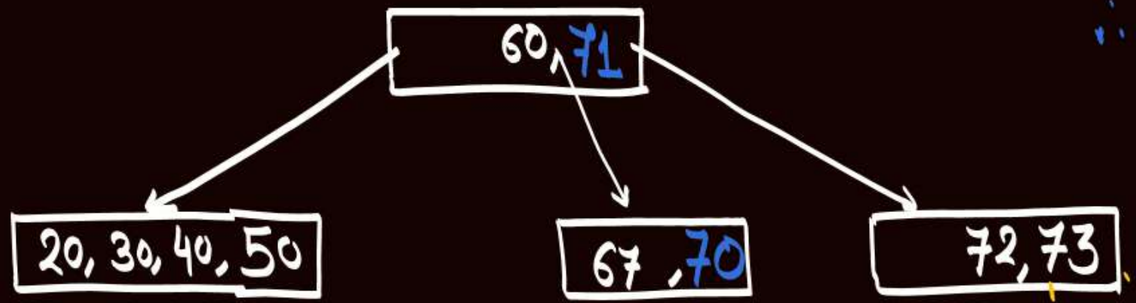
65, 53, 66, 52, 60

H.W. Q: Consider following B tree of order = 5.



⇓ delete (52)

{ Already at leaf node, but leaf node contain Min no of Key: { None of the sibling can help ∴ merge with one of its sibling along with one key from parent



order = 5

Min. of key a non-root node must have = $\lceil \frac{5}{2} \rceil - 1 = 2$

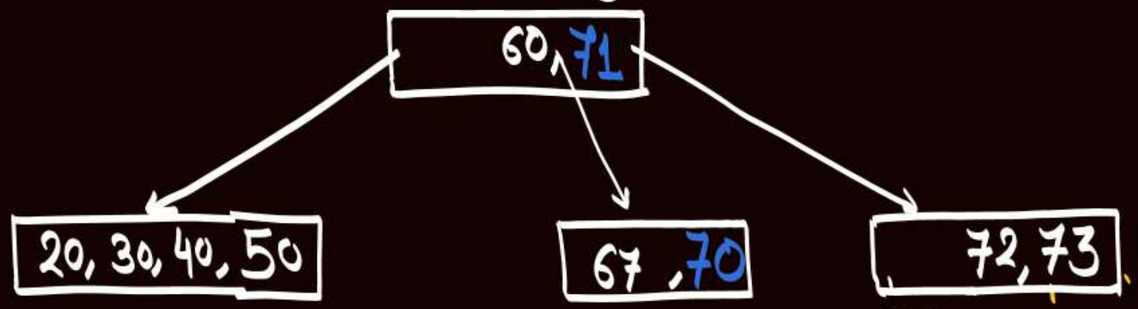
Min. of key a root node must have = 1

Delete the following keys from above B tree in the same order,

65, 53, 66, 52, 60

H.W.

Q: Consider following B tree of order = 5.



⇓ delete (60)

{ 60 is at internal node,
i: Swap with inorder
Predecessor (or Successor)

Swap with key from the node that contain more no. of keys

order = 5

Min. of key a non-root node must have = $\lceil \frac{5}{2} \rceil - 1 = 2$

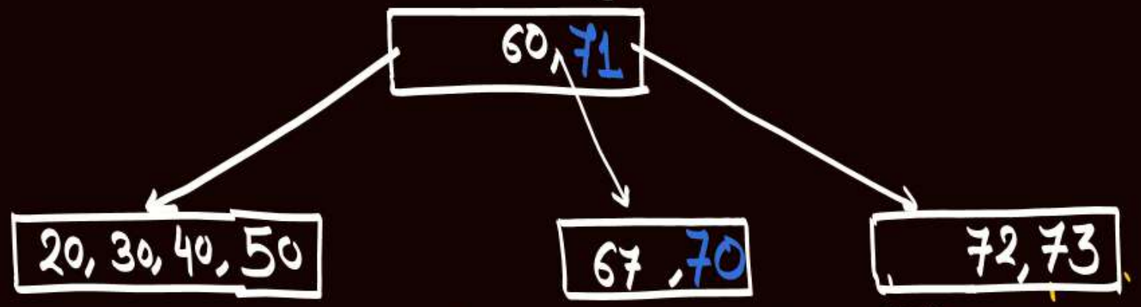
Min. of key a root node must have = 1

Delete the following keys from above B tree in the same order,

65, 53, 66, 52, 60

H.W.

Q: Consider following B tree of order = 5.

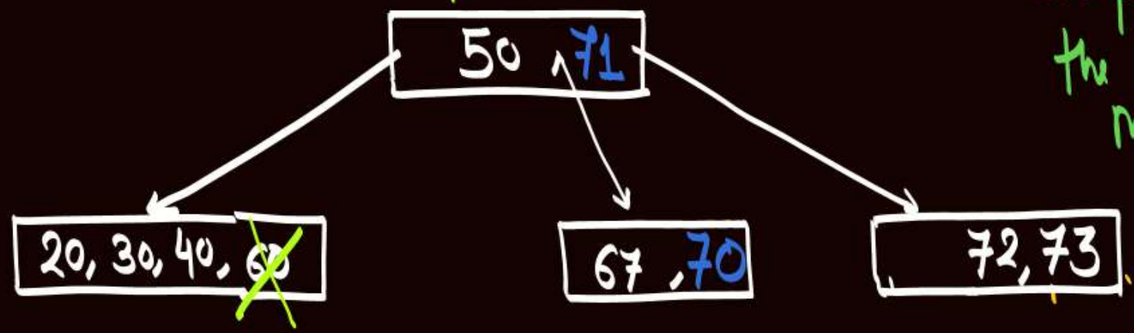


⇓ delete (60)

{ 60 is at internal node,
i.e. Swap with inorder
Predecessor (or Successor)

Swap with key from the node that contain more no. of keys

Swap 60 with inorder Predecessor (i.e. 50) and delete from leaf node



order = 5

Min. of key a non-root node must have = $\lceil \frac{5}{2} \rceil - 1 = 2$

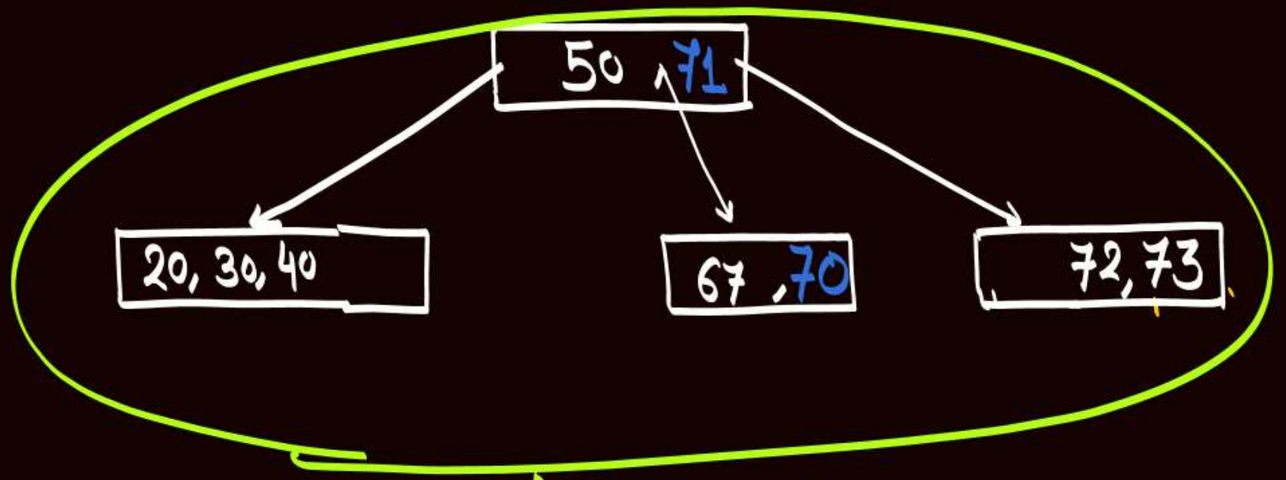
Min. of key a root node must have = 1

Delete the following keys from above B tree in the same order,

65, 53, 66, 52, 60

H.W.

Q: Consider following B tree of order = 5.



Final B tree after all the deletion w.r.t. sequence

order = 5

Min. of key a non-root node must have = $\lceil \frac{5}{2} \rceil - 1 = 2$

Min. of key a root node must have = 1

Delete the following keys from above B tree in the same order,

65, 53, 66, 52, 60

Insertion & deletion w.r.t. B+ tree



Topic : Insertion in B+ tree

Insertion will be performed at leaf node, therefore search for appropriate leaf node that can hold the key to be inserted.

1. If leaf node is not full then insert the key into the leaf node in ascending order.
2. If leaf node is full,
   a) Insert the key in the ascending order.
   b) Split the node into two parts, and copy the median position key into the parent node, Because of this if there is overflow in the parent node then split the parent node as well.

because all the keys must be present at leaf node as well

{while splitting the parent node the median position key is promoted (not copied) to its parent node
{Because keys in the internal node are just dummy keys}

#Q. Let,

Order of internal node of B+ tree = 3, and

Order of leaf node of B+ tree = 2

Insert 5, 15, 25, 35, 45 into the B+ tree in the same order

∴ Max no. of child pointer = 3
Max no. of keys = 3 - 1 = 2
Min no. of child ptr = $\lceil \frac{3}{2} \rceil = 2$
Min no. of keys = $\lceil \frac{3}{2} \rceil - 1 = 2 - 1 = 1$ } Already same for root

∴ Max No. of keys in leaf node = 2
Min no. of keys in leaf node = $\lceil \frac{2}{2} \rceil = 1$ }

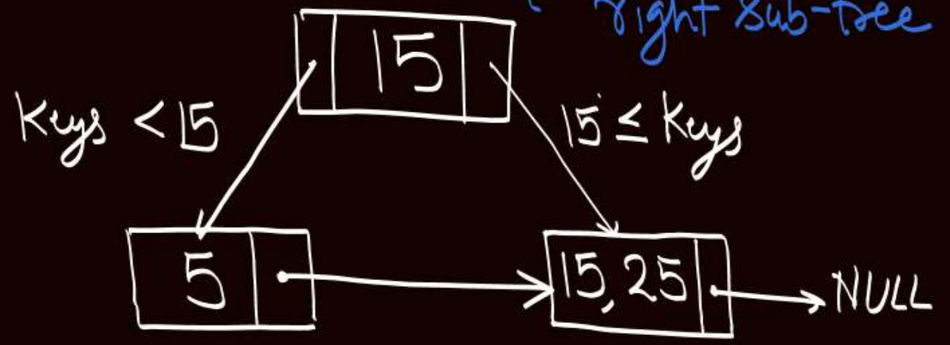
Insert 5, 15, 25, 35, 45

insert (5) ⟹ 

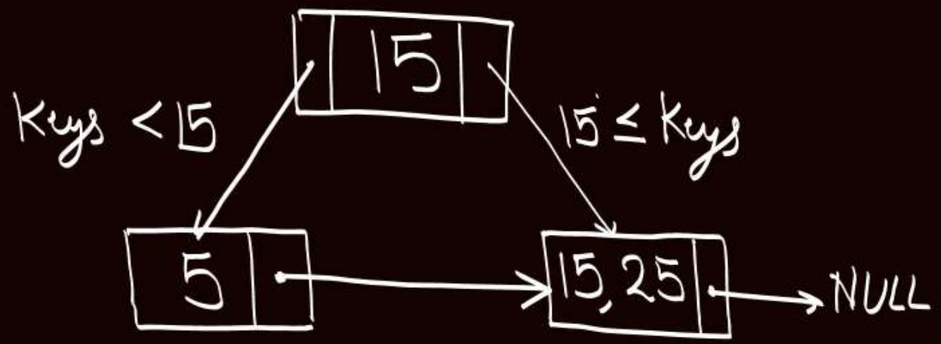
insert (15) ⟹ 

⟹ insert (25) 

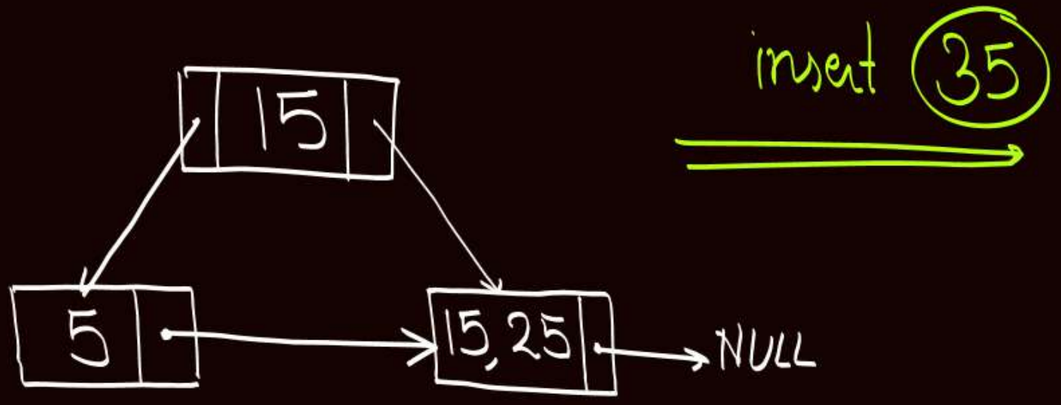
Overflow in leaf node ⟹ ∴ Split



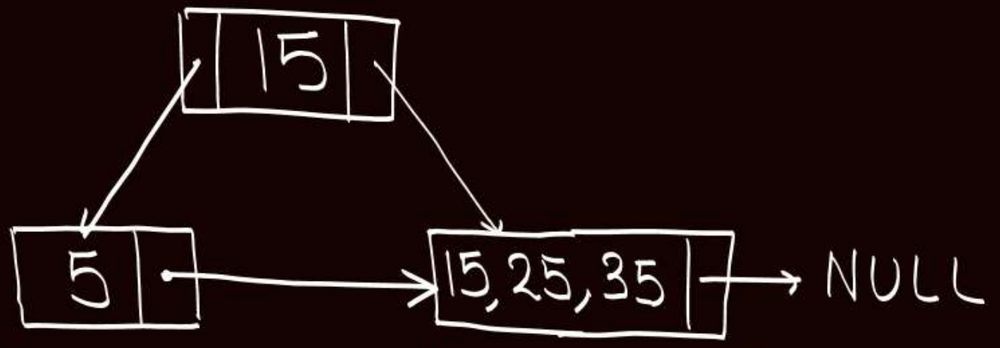
We are assuming key in the Parent node is present in right sub-tree



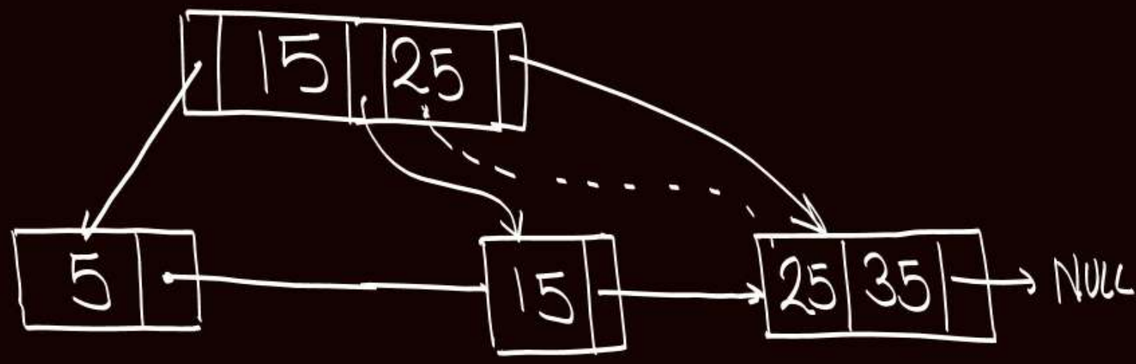
Insert 5, 15, 25, 35, 45



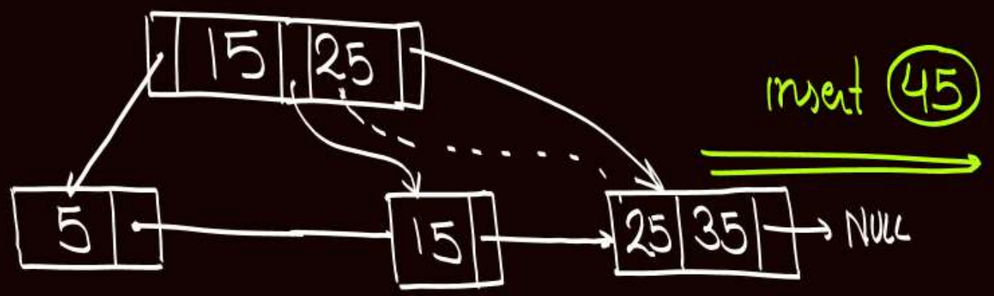
insert 35



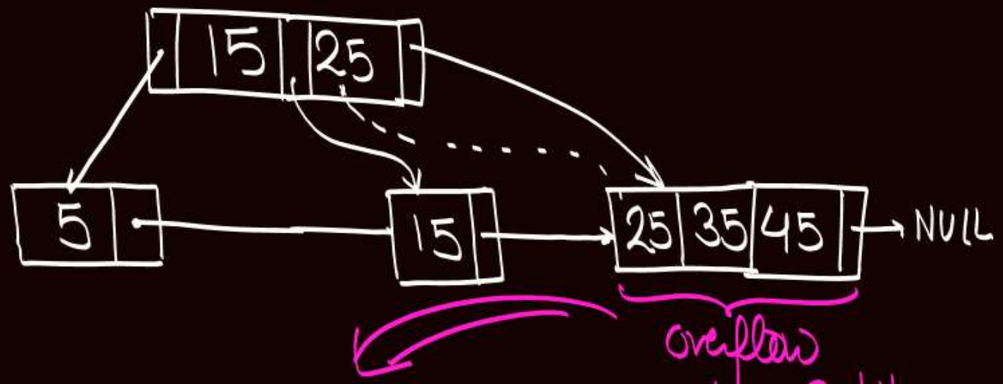
overflow ∴ Split



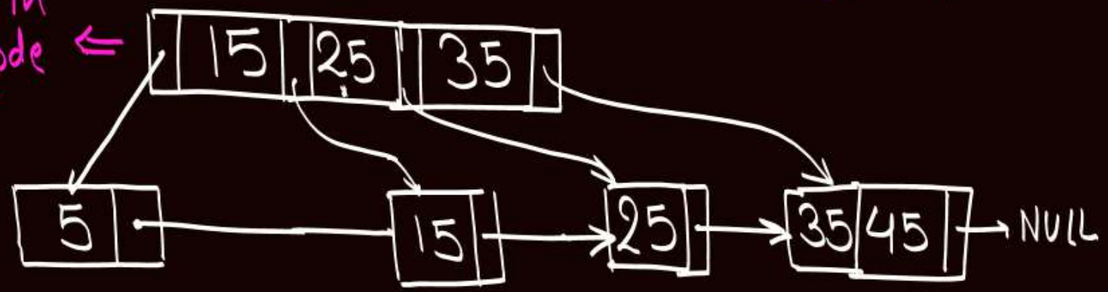
Insert 5, 15, 25, 35, 45

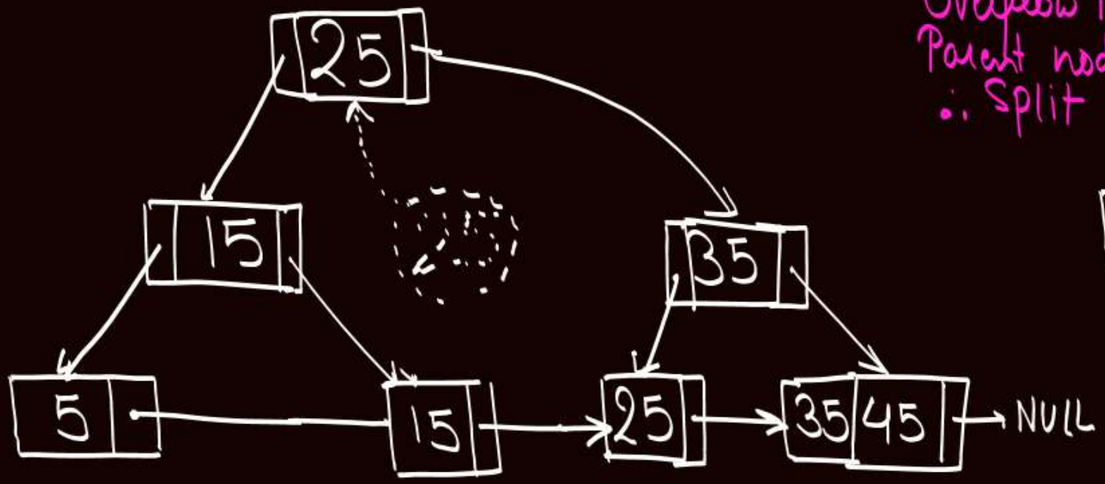


insert (45)



overflow in Parent node ∴ split ⇐







Topic : Deletion in B+ tree

While deleting a key from B+ tree there are two different cases.

Case 1 : When key to be deleted is present only at a leaf node.

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (Keys in the internal node are just dummy keys)



Topic : Deletion in B+ tree

Case 1 : When key to be deleted is present only at a leaf node.

1. If leaf node contains more than the minimum number of keys required, then simply delete the key.

$\#keys = \lceil \frac{m}{2} \rceil$

2. If leaf node contains exact minimum number of keys required, then delete the key and borrow from the immediate siblings if they can help (left then right order).

(redistribute) {When we borrow from the sibling then key in the parent node needs to be updated in order to preserve the search property}

3. If siblings can not help then merge with one of its sibling. Because of this merging if parent node becomes deficient then parent node will borrow from its immediate siblings (if possible) or merge with one of its siblings, it can go up to root node.

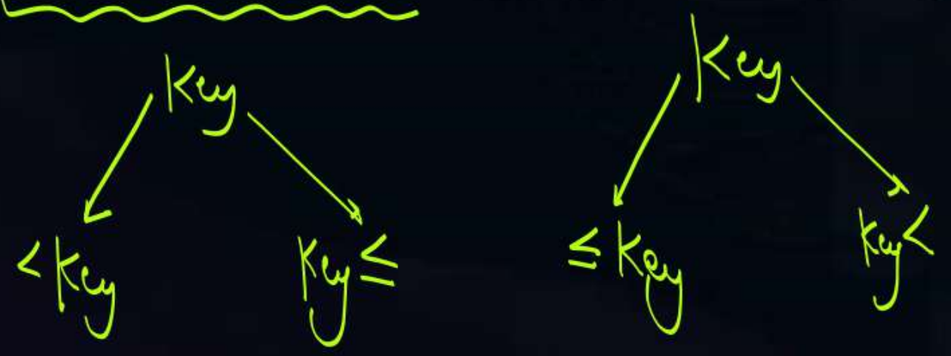
When sibling can not help.



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

1. If leaf node contains more than the minimum number of keys required, then simply delete the key from leaf node as well as from internal node. Because of deletion from internal node an empty space will get created, fill that empty space of internal node with in-order successor (or predecessor)





Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

2. If leaf node contains exact minimum number of keys required and any of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node, borrow the key from the sibling and fill that empty space of internal node with in-order successor (or predecessor)



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

3. If leaf node contains exact minimum number of keys required and none of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node and merge the leaf node with one of its sibling, it may create deficiency in the parent node therefore repeat the process for parent node as usual. {it may go upto root node}

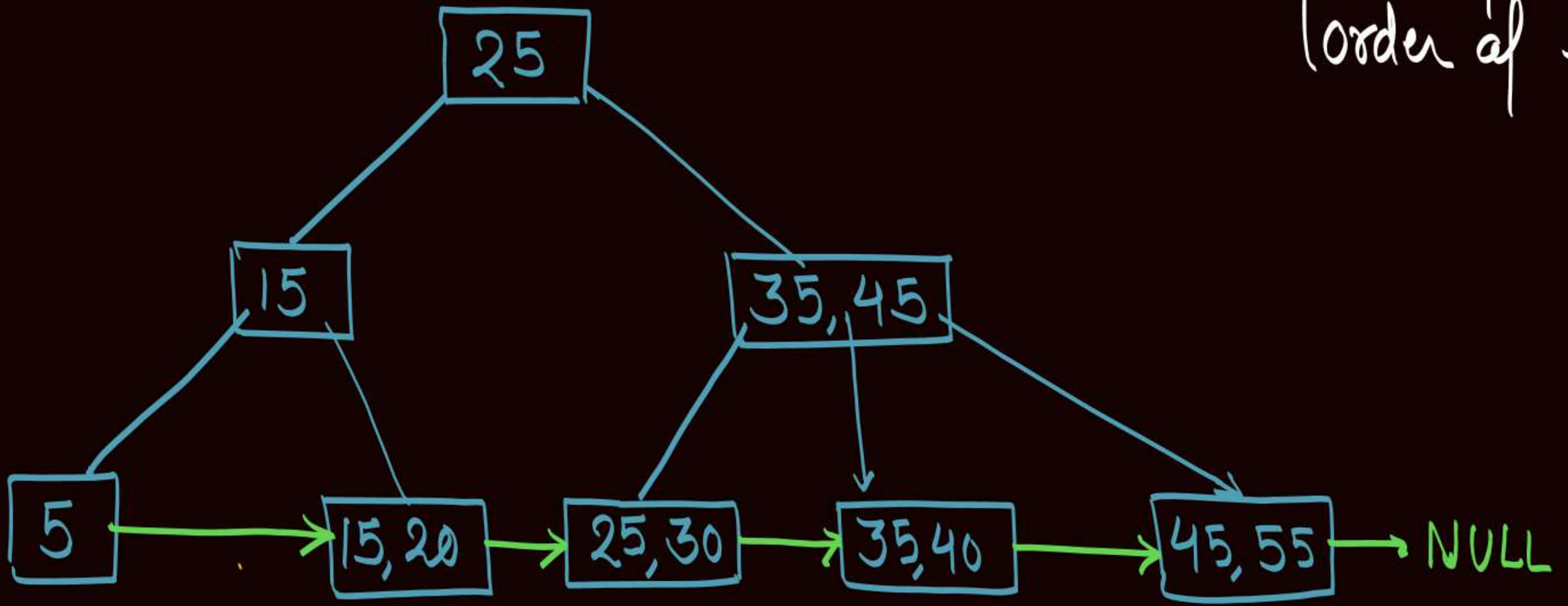
If empty space created in internal node is not handled while merging the parent node then fill it with in-order successor (or predecessor).

H.W.

Q:-

Consider the following B+ tree

Let,
order of internal node = 3
order of leaf node = 2



Delete the following elements from the above B+ tree in the same order

40, 5, 45, 35, 25, 55



2 mins Summary

- **Topic** Deletion from B tree
- **Topic** Insertion into B+ tree
- **Topic** Deletion from B+ tree


THANK - YOU
Slide